Slot-Chain: A Preconfirmation Protocol

Production Design Candidate — v2.1

August 2026

Abstract

Slot-Chain assigns one-second L2 slots to bonded builders. Builders sign parent-linked blocks off chain; any party can prove and submit a batch to L1. Normal candidates compete for a fixed settlement interval. This revision adds a builder-independent recovery lane: after an objective finality breach, or when a forced L1 message becomes due, anyone can prove an unsigned deterministic escape block. It also fixes the exact schedule encoding, historical-state authentication, candidate data binding, EVM transition boundaries, resource bounds, reorg semantics and migration state. The result is an implementation-ready consensus specification, but not a production authorization: the proving, gas, economics and adversarial-test gates in §13 must pass before launch.

Contents

1 Status, claims and exclusions

Normative words MUST, MUST NOTand SHOULDhave their usual RFC meaning. Where prose and either executable model disagree, deployment is blocked until the disagreement is resolved; neither source silently overrides the other.

1.1 Claims

Subject to L1 safety/liveness and a sound validity-proof system, the protocol claims:

1. **Finalized-state safety.** L1 accepts only a proved state transition extending the stored canonical tuple. Builder signatures never substitute for execution validity.
2. **Bounded protocol recovery.** A builder cartel, an absent aggregator, an empty builder registry, and missing standbys cannot prevent an unsigned escape candidate from advancing the canonical state.
3. **Forced transaction inclusion.** A correctly prepaid L1 envelope at the queue head is consumed by the next canonical block after it becomes due; it is executed if valid and consumed-and-discarded if invalid.
4. **Permissionless landing and recovery.** No allowlist, builder key, aggregator seat or payment recipient is required to submit a proof or an escape candidate.
5. **Deterministic lookahead.** Every implementation derives the same schedule from an authenticated finalized snapshot and the byte encoding in §4.

1.2 Non-claims

- A preconfirmation is not finality or insurance. Before normal close it may be revoked by builder equivocation, an unprovable skip, a withheld body, or an escape commit. Applications must price this as probabilistic/economic assurance and publish their own exposure limit.
- A bond cannot cap aggregate off-chain promises: the protocol has no reservation ledger and a builder can sign mutually exclusive promises. `L_LEASE` is deterrence and a recovery source, not guaranteed coverage.
- Protocol liveness is a capability claim. Economic liveness additionally assumes at least one actor can fund proof generation and an L1 transaction. A payment failure never invalidates an otherwise valid canonical transition.
- Censorship by L1 for longer than `T_INCLUDE_MAX`, failure of L1 finality, or a broken proof system is outside the model.

2 Actors and trust model

Builder.
A registered address eligible for scheduled slots. Builders may equivocate, skip parents, withhold bodies, collude, or disappear.

Aggregator.
The current auction seat, expected to collect data, prove and submit normal candidates. It has no exclusive consensus authority.

Prover/lander.
Any address. It supplies a validity proof and names a pull-payment beneficiary. Proof validity, not identity, authorizes transition.

Forced-message sender.
Any L1 account that submits a gas-bounded, prepaid L2 transaction envelope.

L1.
Ethereum provides canonical ordering, timestamps, EIP-4788 beacon roots, EIP-2935 historical block hashes, blob commitments and the KZG point-evaluation precompile.

The safety threshold contains no honest-builder assumption. Honest builders improve normal throughput and preconfirmation reliability; the unsigned escape lane provides state and forced-queue progress when all of them collude. The L1-liveness assumption includes transaction inclusion and at least 64 canonical L1 blocks becoming available within `T_DEPTH_MAX=900s`; a longer L1 stall suspends, rather than violates, the L2 recovery clock.

3 Clock domains and canonical state

L2 slot zero is fixed by `GENESIS_TIMESTAMP`, with

`currentL2Slot=block.timestamp-GENESIS_TIMESTAMP.`

L2 slot and every deadline are timestamp seconds. L1 confirmation depth is counted in L1 block numbers. Beacon slot is used only in schedule authentication. Implementations MUST NOTsubstitute one clock for another.

The L1 contract stores the following canonical tuple:

```
Canonical = (tipHash, tipSlot, stateRoot, messageCursor,
             canonicalizationBlock, canonicalizationHash, winningDataRoot)
```

`canonicalizationBlock` is the L1 block that most recently committed the tuple. A recovery candidate authenticates its block hash through EIP-2935 and requires depth `F_L1`. An anchor block is authenticated the same way. The initial values and the bridge cursor are migration inputs described in §11.

Every state-mutating external entry point begins with `sync()`. If `sync()` changes mode or commits a mature normal candidate, the wrapper MUSTreturn `SYNCED` successfully before parsing or validating the caller's candidate. The caller resubmits against the new version. This rule is necessary because a later Solidity revert would otherwise roll back the sync transition in the same transaction.

4 Builder registry and exact lookahead

4.1 Registry lifecycle

A registration contains a nonzero address, a `uint192` bond not exceeding `BOND_MAX`, a monotonic `uint64registrationIndex`, and separately reserved `L_LEASE[window]` tranches. Address zero is permanently reserved for `VACANT`. Duplicate live addresses and re-registration of a tombstoned key are forbidden.

The active table has exactly 64 indexed cells. Admission into a full table atomically replaces only its minimum-bond live entry after the delayed-entry rule; displaced entries retain historical liabilities. Because the complete table is bounded and authenticated, snapshot verification proves completeness rather than accepting an unverifiable claimed "top 64." Every registry mutation updates a fixed-depth Keccak `registryRoot` over all 64 cells.

Eligibility is evaluated before ranking: an entry is eligible for window W only if its registration was effective in the authenticated snapshot, it had a fully reserved tranche for W, and its tombstone was not effective at that snapshot. The 64 eligible entries with greatest bond, then smallest registration index, form the snapshot set. If none exist, every ticket is `VACANT`. A later slash tombstones the key atomically and turns its already-snapshotted future tickets into gaps; it is excluded from all later snapshots.

A tranche follows the finite state machine `FREE->RESERVED(W)->LIABLE(W)->RELEASED` or `RESERVED/LIABLE->SLASHED`. Candidate acceptance marks every used window liable. Release requires the window to be below `minAdmissibleSlot`, its normal/recovery and L1 reorg horizons to have passed, and the evidence delay to have expired. A bitmap per snapshot and a ring of at most `MAX_LIVE_WINDOWS` replace unbounded slash-history arrays.

4.2 Authenticated snapshot

For aligned window W, let its start be $384W$ and let the target beacon timestamp be five L1 epochs earlier. Anyone calls `sealWindow(W)` after the source has `F_FINAL` depth and before EIP-4788 history expires. The contract performs the following deterministic procedure:

1. Starting at the target timestamp, query EIP-4788 at twelve-second steps backward and choose the first nonzero root, for at most `MAX_SNAPSHOT_MISSES=64` steps. This is the greatest canonical beacon block at or before the target within the assumed L1-liveness bound. No result means an all-`VACANT` window.
2. Verify SSZ generalized-index branches from that beacon block root to the execution payload's `blockHash`, `stateRoot` and `prevRandao`. These three values therefore come from one payload.
3. Verify Merkle–Patricia storage proofs from that execution state root for the registry header and `registryRoot`. The caller supplies all 64 canonical serialized cells, including empty cells; recompute their six-level Keccak tree and require equality to `registryRoot`. Filter eligibility and order the entries, then store the entry-root and seed. The fixed leaf count proves completeness with one MPT path rather than 64 independent storage paths. The execution block hash from the SSZ payload is also authenticated through EIP-2935 at `F_FINAL` depth.

`sealWindow` is permissionless and pays a fixed pull credit from a pre-funded maintenance reserve. A missing seal fails closed to `VACANT`; it cannot give an address an unauthorized ticket. The snapshot age plus finality delay is less than EIP-4788's 8,191-slot history window.

4.3 Consensus byte encoding and allocation

All hashes use Ethereum legacy Keccak-256. Concatenation below is packed fixed width; strings are the literal ASCII bytes without a length prefix:

$$
\begin{aligned}
seed_W &= H(\texttt{"slot-chain-seed-v1"} \| u256(chainId) \\
&\quad \| u256(protocolVersion) \| u256(W) \| bytes32(prevRandao)), \\
qTie_i &= H(\texttt{"slot-chain-quota-tie-v1"} \| seed_W \| u64(regIndex_i)), \\
sTie_{p,i} &= H(\texttt{"slot-chain-slot-order-v1"} \| seed_W \| u16(p) \| address20(i)).
\end{aligned}
$$

Starting from zero, capped D'Hondt awards each of 384 tickets to the unsaturated entry maximizing `bond/(allocation+1)`, with `Q_MAX=76`. Quotients are compared by cross multiplication; a `uint192` bond times `Q_MAX+1` cannot overflow `uint256`. Equal quotients use smallest `qTie`. Remaining tickets are `VACANT`; six addresses are required to fill a window.

Placement processes positions in order. A real address is feasible if it has a ticket remaining, differs from the preceding real address, and removing it preserves `remaining[a] <=otherRemaining+1` for every real address. Choose the feasible value with smallest `sTie`; `VACANT` may repeat. The executable model and its golden vectors are normative test fixtures.

5 Blocks, promises and proof statement

5.1 Signed header

A builder signs EIP-712 typed data over at least:

```
(chainId, protocolVersion, verifyingContract, slot, parentHash,
blockHash, stateRoot, bodyRoot, anchorNumber, anchorHash,
forceRoot, forceCutoff, messageStart, messageEnd, dataManifestRoot,
 beneficiary,
 tier, episodeVersion, activationId, finalRefNumber, finalRefHash)
```

The normative EIP-712 type string is:

```
SlotChainBlock(
 uint256 chainId,uint256 protocolVersion,address verifyingContract,
 uint64 slot,bytes32 parentHash,bytes32 blockHash,bytes32 stateRoot,
 bytes32 bodyRoot,uint64 anchorNumber,bytes32 anchorHash,bytes32 forceRoot,
 uint64 forceCutoff,uint64 messageStart,uint64 messageEnd,
 bytes32 dataManifestRoot,
 address beneficiary,uint8 tier,uint64 episodeVersion,
 bytes32 activationId,uint64 finalRefNumber,bytes32 finalRefHash)
```

Typography-only line breaks and indentation are removed without inserting spaces; ordinary spaces displayed inside fields are retained before hashing. `TYPEHASH` is legacy Keccak-256 of those ASCII bytes and equals the concatenation of these two fragments:

```
0x9388dcc807c27fa0c6b56a0cdcf071d9
f0e250a9855faec7749a35d1a5e5bfa7
```

The EIP-712 domain is `("SlotChain","2",chainId,verifyingContract)`. Signatures use low-s secp256k1 and reject zero/recovered-zero addresses. The execution `blockHash` excludes the off-chain builder signature, avoiding a circular hash definition.

5.2 Three validity tiers

1. **Normal signed.** Every block is signed by its scheduled, non-tombstoned builder. Slots strictly increase, each slot is no later than `currentL2Slot+CLOCK_SKEW`, and the first slot strictly exceeds the canonical base slot. Parent gaps are at most `G_MAX=64`.
2. **Recovery signed.** The first block extends the L1-final canonical tuple, binds the active episode, has slot at least `activationSlot`, and may cross an unbounded time gap. Every block still requires its scheduled builder signature.
3. **Escape unsigned.** Exactly one deterministic block extends the L1-final canonical tuple during recovery. It has no builder signature, no beneficiary-controlled coinbase, no discretionary transaction and no blob body. It contains only the protocol anchor and the deterministic maximum prefix of the activation-frozen forced queue. When the queue is empty an SLA-triggered episode may commit an empty anchor-only escape block. Its slot is `max(activationSlot+ESCAPE_OFFSET,canonical.tipSlot+1)` and its anchor, queue root/cutoff, block hash and state root are unique functions of the episode and proved execution result.

The circuit verifies parent linkage, strict slot monotonicity, schedule and tombstone state, signatures for tiers 1–2, episode binding for tiers 2–3, historical references, anchor causality, the forced prefix, execution, exact body/data coverage and the public outputs. It rejects a candidate with more than 4,096 blocks, 12 schedule windows or 2,100 referenced blob records.

5.3 Preconfirmation semantics

A builder signature proves authorship and makes same-slot equivocation slashable. It does not prove that the body reached a prover or that the next builder saw it. A later scheduled builder may sign a profitable skip that the protocol cannot distinguish from non-receipt. Accordingly:

- before L1 candidate acceptance, a promise is *observed*;
- after acceptance and before close, it is *provisionally proved*;
- after normal close or recovery commit, it is *canonical*; and
- only the underlying L1 finality policy makes it *L1-final*.

Wallets and applications MUST NOTdisplay the first two states as final. An escape commit explicitly revokes every omitted observed or provisionally proved promise and returns omitted transactions to the mempool.

6 Blob data authentication

The previous six-blob-per-candidate rule is removed: it contradicted a candidate that may span thousands of one-second blocks. Data is instead posted into a publisher-owned, bonded session. An attacker can fill only its own session; expired sessions are garbage-collected and their storage deposit pays the collector.

For every blob attached to `postData(session,manifests[])`, the contract:

1. obtains `versionedHash=BLOBHASH(i)` from the same transaction;
2. computes the following value, reduced to the KZG field:

```
z = keccak256("slot-chain-data-fs-v1", chainId, protocolVersion,
              session, versionedHash, bodyRoot, publisher, validUntil)
```

3. verifies the supplied `(z,y,KZGProof)` against that versioned hash with the point-evaluation precompile; and
4. appends `(index,versionedHash,bodyRoot,publisher,validUntil,z,y)` to a Keccak Merkle-mountain-range whose root commits to its leaf count.

The candidate proof receives the blob polynomials as private witness, recomputes each polynomial evaluation $P(z) = y$, decodes the canonical SSZ body chunks and recomputes `bodyRoot`. It proves MMR membership and that the candidate manifest uses increasing unique indices and partitions every discretionary block body exactly once. A record not in the manifest is irrelevant; a manifest duplicate, extra chunk or uncovered byte is invalid. At submission the session root and count must equal the proof's public inputs.

This split gives constant L1 verification per `postData` call and a bounded proof statement for a whole candidate. `validUntil` must extend past candidate close, proof time and the L1 reorg margin. The initial 2,100 record cap exceeds the physical maximum of six Ethereum blobs per L1 block over a 4,096-second candidate; the proof benchmark must confirm that the chosen aggregation implementation meets `P_PROVE_MAX`.

Data publication has no consensus-coupled reimbursement. The normal service contract may pay it, and a forced sender prepays its L1 envelope, but a failed or empty payment vault cannot revert a canonical commit.

7 Settlement and recovery state machine

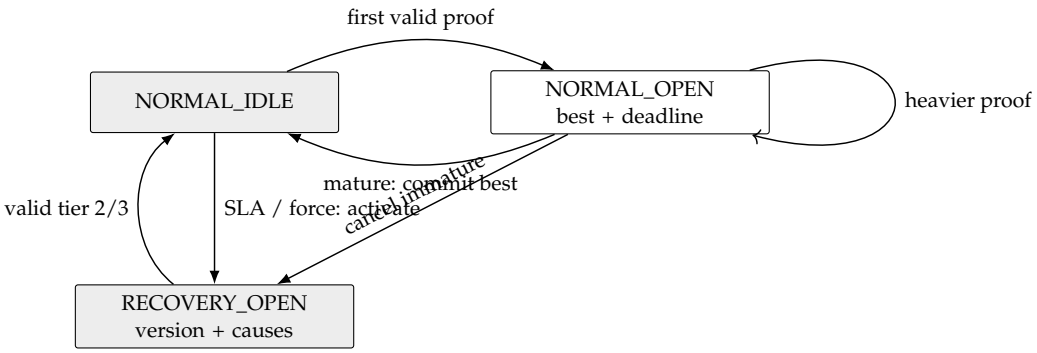


Figure 1: The complete mode machine. Payments are outside every arrow.

7.1 Normal candidates

The first valid tier-1 candidate extending the canonical tuple opens a normal window. It freezes `baseCanonical`, `deadline=block.timestamp+W_SETTLE_SECONDS`, and `minAdmissibleSlot=currentL2Slot-DELTA_TIP`. Later candidates use the same base, deadline and minimum. Their total order is lexicographic:

$$(\text{block count descending},\ \text{tip slot descending},\ \text{tip hash ascending}).$$

Only a strictly better candidate replaces `normalBest`; provisional replacement changes no

canonical state and pays nobody. At or after the deadline, `sync()` atomically commits exactly the stored best. It then recomputes recovery causes against the new canonical tuple.

7.2 Activation

While normal, `sync()` opens one persistent recovery episode if either:

- `currentL2Slot-canonical.tipSlot>DELTA_FINAL_LAG`; or
- the forced queue head satisfies `enqueuedAt+FORCE_DELAY<=block.timestamp`.

A mature normal window closes first. Any immature `normalBest` is canceled unconditionally; it is never promoted across activation. The contract increments `episodeVersion`, freezes `activationSlot`, and derives:

```
activationId = keccak256(domain, chainId, contract, version,
                         activationBlock,
                         blockhash(activationBlock-1), causes)
```

It also freezes `forceRoot`, `forceCutoff`, the preceding L1 block as `escapeAnchor`, and the deterministic `escapeSlot`. Queue appends after activation belong to a later block and cannot invalidate an in-flight escape proof. SLA cause terminates and burns the incumbent once, then promotes an eligible standby or leaves the seat empty. Force-only cause does not punish the seat. Missing seats and escrows are valid states.

7.3 Recovery acceptance

Recovery is first-valid, not heaviest. A tier-2 or tier-3 candidate MUST:

1. extend the stored canonical tuple and authenticate its `canonicalizationHash` through EIP-2935 at depth `F_L1`;
2. bind the current version and activation ID;
3. have first slot at least `activationSlot`, tip no older than `DELTA_TIP`, and no slot later than wall clock plus skew;
4. consume the exact maximum forced-message prefix from the current cursor, bounded by the activation-frozen cutoff; and
5. satisfy its tier-specific signature/data restrictions and validity proof.

The first valid L1 transaction atomically commits the tuple, records the current L1 block/hash, clears the episode and data session, and returns to normal. Later proofs are stale. Candidate rejection cannot undo activation because of the early-return wrapper in §3.

8 Forced queue and unsigned escape

A forced envelope is `(sender,nonce,rawTx,gasLimit,maxFee,refundAddress)`. Admission requires `gasLimit<=MAX_FORCE_MESSAGE_GAS`, a byte cap, and a deposit covering worst-case L2 execution, escape proof credit and L1 storage. The queue is FIFO and append-only. The raw bytes are permanent L1 calldata committed by an on-chain Keccak hash; blobs are not an alternative for this availability path. The per-envelope byte cap is 131,072 bytes.

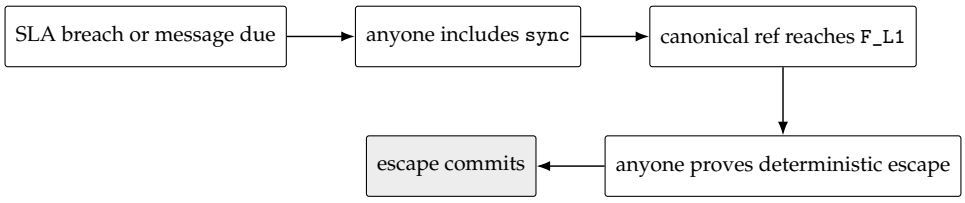
Each proved block consumes the maximum prefix whose declared gas limits fit `FORCE_GAS_BUDGET`. Admission guarantees the head fits, so a due head always advances. Execution uses ordinary L2 nonce/balance rules. A revert, bad nonce, insufficient balance or expired user intent consumes and discards the envelope; otherwise an invalid head could deadlock every later sender.

For a signed block, `forceRoot/forceCutoff` are the queue state authenticated by that block's L1 anchor, and the circuit consumes the maximum prefix only up to that cutoff. A later L1 append therefore cannot invalidate an already signed block or an in-flight proof. Cutoffs are nondecreasing along a candidate. A due current head still cannot be bypassed at submission: `sync()` opens a recovery episode before normal candidate validation.

The ten-minute force delay applies to the queue head, not to an arbitrary position behind existing prepaid work. An admitted sender can compute its worst-case number of escape blocks from the gas ahead at admission. As with L1 itself, the protocol does not promise backlog-independent latency against an adversary willing to buy all available capacity; it promises that builders cannot selectively veto the FIFO head.

An escape block is deterministic. Its timestamp is its frozen slot, coinbase is the protocol sink, base fee follows the ordinary formula, randomness is derived from the frozen authenticated anchor, and its only transactions are anchor plus the frozen forced prefix. The circuit proves those facts. No party gains timestamp, ordering or MEV discretion by producing the proof.

For a cartel with no usable builder block, the recovery sequence is:



With the initial bounds, activation inclusion (120 s), the frozen `ESCAPE_OFFSET=1,900s` (covering `T_DEPTH_MAX=900s`, proving at 900 s and 100 s margin), submission inclusion (120 s), and clock margin (24 s) total 2,164 seconds, below `DELTA_FINAL_LAG=3,600s`. Tip admissibility is separate: the escape slot is at the end of the offset, so only submission and skew—144 seconds—must fit `DELTA_TIP=1,200s` when proof generation meets its bound.

9 Economics, slashing and payments

Event	Objective evidence	Effect
Same-slot equivocation	Two valid distinct signed headers for one key/slot	Slash that schedule window's tranche once; tombstone key; fixed reporter pull credit.
Aggregator SLA breach	Final lag exceeds threshold after mature close	Terminate and burn penalty bond once; open recovery; promote standby if any.
Force-only activation	Due queue head	No aggregator penalty; open recovery.

Skip or absence	None that distinguishes fault from non-receipt	Not slashable; promise may be revoked.
Invalid proof/data	Verifier failure	Reject; no canonical or payment effect.

Equivocation evidence is idempotent per `(builder,window)`; separate windows slash separate tranches. `L_LEASE` is calibrated against at most 76 assigned slots but is not insurance for unbounded signed value. Tombstoning is atomic with the first valid slash; future snapshots exclude the key.

All rewards are pull payments recorded after a successful transition. A proof beneficiary may claim at most the episode's funded cap from, in order, the forced-envelope deposit, the terminated seat's dedicated recovery escrow, and an optional protocol reserve. Exhaustion records zero claimable amount and does not revert. Burns are never recycled into the same transaction's bounty. Normal service/data compensation belongs to the auction/service contract and cannot affect candidate validity or canonical commit.

10 Security and liveness analysis

Adversary/failure	**Protocol result**	**Bound or residual**
Invalid execution or forged state	Proof rejects.	Finalized-state safety reduces to proof-system and L1 safety.
Aggregator offline/byzantine	SLA activation terminates it; any prover uses tier 2 or 3.	At most final-lag threshold plus 2,164 s under stated inclusion/proof bounds.
No seat or no standby	State remains valid; escape has no seat dependency.	Same protocol bound; reward may be zero.
All builders collude/absent	Unsigned escape advances anchor and forced queue.	Meaningful discretionary throughput falls to forced envelopes; no builder censorship veto.
Builder equivocation	Competing signed branches; one may win normal order.	Direct per-window slash; preclose promises remain revocable.
Builder skips/withholds body	Honest tail may be unprovable or omitted.	Not objectively slashable; eventual escape restores state progress but may revoke promises.
Data-session spam	Sessions are publisher-owned and bonded.	Attacker fills only its session; escape uses no blob session.
Transparent-mempool front-run	First valid recovery transaction wins.	Candidate binds beneficiary, episode and exact outputs; copying proof cannot redirect payment or state.
Payment-vault exhaustion	Claim is zero or partial.	Never blocks safety or canonical liveness; economic liveness is conditional.

L1 reorg within policy	Contract state, burns and claims roll back together.	Clients replay canonical logs and proofs against new version/hash.
L1 censorship/outage	Neither activation nor proof can land.	Outside model after `T_INCLUDE_MAX`; no L2 protocol can force its L1 contract to execute.

The escape lane establishes strong transaction-entry permissionlessness even when the capital-ranked builder set is captured. It does not make the normal builder market egalitarian: top-64 ranking and per-window capital remain a meaningful entry barrier. The product must distinguish these two claims.

11 Implementation blueprint

11.1 L1 modules and bounded storage

- `BuilderRegistry`: entries, tranche bitmap, tombstone and bounded liability ring.
- `ScheduleOracle`: sealed window root/seed in a ring covering `H_LOOK+MAX_CANDIDATE_WINDOWS+reorgMargin`; old entries are reusable.
- `DataSession`: one bonded session per owner nonce, 2,100-leaf MMR frontier, count and expiry; no on-chain leaf array.
- `ForcedQueue`: append-only envelope commitment, cursor and deposits; payload bytes use L1 calldata/blob storage according to the bridge profile.
- `Settlement`: canonical tuple, one normal best, one recovery episode, seat pointer and pull-payment balances. No loop is proportional to chain age.

All setters enforce parameter inequalities atomically and are disabled after the launch freeze except through the existing delayed governance path. Every counter has an explicit width and checked increment; no sentinel shares a legal value.

11.2 Historical authentication

EIP-2935 is the only source for recent execution block hashes. Recovery MUST NOTuse Solidity's 256-block `blockhash` limit for the 64-block depth plus proving path. EIP-4788 plus SSZ branches authenticates the schedule's beacon/execution payload; the payload state root plus MPT proofs authenticates the registry. Deployments lacking either primitive require an equivalently verified oracle and a separate security review.

11.3 Reorg rule

The canonical L1 log is the journal. A reorg truncates and replays, in order: canonical tuple, mode/version, normal best/deadline/frozen minimum, activation, seat termination/burn, tombstone/tranche state, forced cursor, data-session root/count, and pull credits. Off-chain workers index by `(chainId,contract,blockHash,logIndex)` and discard orphaned jobs. Withdrawal/bridge execution waits for the configured L1 finality policy.

11.4 Genesis and migration

Deployment occurs at least `D_SNAP+F_FINAL+sealMargin` before `ACTIVATION_TIMESTAMP`, so every first live schedule source is post-deployment and sealable; there are no fabricated pre-genesis snapshots. At a finalized migration checkpoint, governance commits the old canonical tip, state root, message queue root/cursor and bridge withdrawal cursor to the new contract. The new verifier accepts no pre-activation candidate. The old entry point rejects slots at or after activation. A release rehearsal proves that no slot is accepted by both or neither verifier and that every queued message is represented exactly once.

12 Parameters and enforced geometry

Parameter	Initial value	Enforced relation
L2 slot / schedule window	1 s / 384 slots	Aligned by genesis timestamp.
`N_MAX`, `Q_MAX`	64 / 76	Six addresses fill 384 tickets.
`BOND_MAX`	$2^{192} - 1$	`bond*(Q_MAX+1)<2^256`.
`D_SNAP`, `F_FINAL`	5 / 2 L1 epochs	Source finalized before published lookahead; seal within EIP-4788 history.
`H_LOOK`	768 L2 slots	Guaranteed by snapshot geometry with one-window slack.
`G_MAX`	64 L2 slots	Tier-1 parent gap only.
`W_SETTLE_SECONDS`	1,200 s	Timestamp deadline, never L1 block count.
`DELTA_TIP`	1,200 s	At least submit inclusion + skew = 144 s after the frozen escape slot.
`DELTA_FINAL_LAG`	3,600 s	At least activation + escape offset + submit + skew = 2,164 s.
`F_L1`; `T_DEPTH_MAX`	64 L1 blocks; 900 s	Canonical-reference depth and assumed maximum time to reach it; protected by EIP-2935.
`ESCAPE_OFFSET`	1,900 s	At least depth-time bound + proof bound + margin.
`FORCE_DELAY`	600 s	Maximum wait before force-only activation, excluding L1/proof bounds.
`MAX_FORCE_MESSAGE_GAS`	5,000,000 gas	At most `FORCE_GAS_BUDGET=20,000,000`; head always fits.
Candidate caps	4,096 blocks; 12 windows; 2,100 blobs	Cover final-lag geometry and physical six-blob/L1-block maximum.
`DATA_TTL`	86,400 s	Greater than candidate, settlement, proof and reorg path.
EIP history	8,191 entries	Greater than every referenced recovery path.

Launch tooling evaluates every inequality in both seconds and its native clock unit. Governance cannot set a value that violates a relation.

13 Production gates and verdict

The consensus design is fixed enough to implement. Production deployment is blocked until every gate below produces a versioned artifact and passes:

1. **Proof performance.** Independent tier-1, tier-2 and worst-case tier-3 proofs, including 4,096 blocks and 2,100 data records, finish within 900 seconds on the published minimum hardware. Peak RAM, recursion setup and failure recovery are measured; a miss requires changing parameters and another design review.
2. **Contract gas.** `sealWindow`, `postData`, normal submit, `sync`, recovery submit, slash and garbage collection fit current L1 gas limits with 30% margin. Fuzzing includes empty registry, zero seat, full data session, full queue prefix and maximum history age.
3. **Cryptographic conformance.** Solidity, Go, Rust and circuit code match the Keccak/EIP-712 golden vectors, SSZ generalized indices, MPT proofs, KZG Fiat–Shamir transcript and exact body manifest. Two independent implementations generate every vector.
4. **State-machine verification.** Stateful fuzzing models EVM rollback, same-block ordering, reorg replay, stale versions, activation/close coincidence, message consume-and-discard and storage reclamation. The executable models in Appendix C remain green.
5. **Economics.** Auction bond, per-window tranche, forced-envelope deposit, storage bond and proof credits are calibrated under blob/L1 fee spikes. The published liveness claim explicitly becomes conditional above fee caps.
6. **Operations.** At least two independent prover stacks, public data session mirrors, schedule sealers and recovery bots complete a shadow-fork outage exercise. The exercise removes every builder and seat, forces a user transaction, reorgs the first recovery attempt, and still reaches finality.
7. **External review.** Separate proof-system, Solidity, bridge and economic audits close all critical/high findings; the migration rehearsal and emergency governance path are in scope.

Verdict. *The earlier fallback architecture was not implementation-ready: it depended on a scheduled builder, could revert activation inside a rejected submission, made payment a commit precondition, and could not bind whole-window data with a six-blob cap. This revision removes those dependencies and fixes a single consensus path. It is implementation-ready as a specification. It is not production-ready as a deployed system until the seven empirical gates pass; claiming otherwise from a document and Python assertions alone would be unsound.*

A Normative transition ordering

Every external candidate entry point implements the following structure:

```
function submit(input) returns (Status) {
    if (sync()) return SYNCED;     // successful transaction, never revert
    if (mode == NORMAL) return submitNormal(input);
    return submitRecovery(input);
}

function sync() returns (bool changed) {
    if (normal deadline matured) {
        commit(normalBest);        // if present
        clear normal window;
        changed = true;
    }
    if (mode == NORMAL && (finalLagBreach || forcedHeadDue)) {
        clear any immature normal window;
        open persistent recovery episode;
        if (finalLagBreach) terminate incumbent once;
        changed = true;
    }
}
```

No transfer, reward calculation, unbounded loop or caller-controlled external call is permitted before the canonical write completes. Pull credits are best-effort accounting after the transition.

B Rejected alternatives

1. **Heaviest fallback.** Rejected because an observer cannot prove it has seen the globally heaviest off-chain tail; transparent races can prevent a stable target. Recovery is episode-bound first-valid.
2. **Scheduled-builder-only fallback.** Rejected because the same cartel that stalls normal production also controls recovery. Tier 3 is deterministic and unsigned.
3. **Promote the pre-activation normal best.** Rejected because it was accepted under different cutoff/version semantics. Mature close occurs first; immature state is canceled.
4. **Atomic reward-funded commit.** Rejected because an empty seat or vault becomes a consensus deadlock. Claims are pull-based and non-authoritative.
5. **One same-transaction blob set.** Rejected because Ethereum exposes at most a handful of blobs per transaction while a candidate spans thousands of blocks. Publisher-owned authenticated sessions separate posting from proof.
6. **One KZG opening at a prover-chosen point.** Rejected because it does not soundly tie declared bodies to the blob. The challenge commits to both blob hash and body root, and the circuit recomputes the evaluation and body root.
7. **Arithmetic snapshot slot without missed-slot semantics.** Rejected. The bounded EIP-4788 backward scan selects the greatest available source or fails closed.

8. **Bond as insurance.** Rejected without an on-chain reservation ledger. The tranche is deterrence only.

C Executable consensus models

`lookahead-model.py` implements legacy Keccak, exact-width encodings, snapshot selection, eligibility-before-ranking, capped D'Hondt and feasible placement. It reports 26 assertions, including fixed seed/schedule golden vectors, empty registry, address-zero, tranche, tombstone, overflow and whale regressions.

`settlement-window-model.py` reports 49 assertions over the actual mode machine: early-return EVM semantics, mature-close ordering, no-seat recovery, unsigned escape, exact forced prefix, data binding, slot/history predicates, frozen normal admissibility, reorg replay, resource caps and timing geometry. Cryptographic booleans in that model are not cryptographic evidence; the conformance and proof gates remain mandatory.

D Security invariants checklist

1. Canonical state changes only after one valid proof and only from its stored base tuple.
2. A sync transition cannot be rolled back by validation of the same call's candidate.
3. No payment, seat or standby is required for canonical commit.
4. Recovery candidates bind one live episode and an EIP-2935-authenticated canonicalization block at required depth.
5. A due forced head advances in every canonical recovery block; invalid messages cannot pin the cursor.
6. Tier 3 has no builder signature, discretionary transaction, arbitrary coinbase or blob body.
7. Every tier-1/2 block has a valid scheduled signature, strict slot progress and an authenticated anchor no later than its L2 time.
8. Every discretionary body byte is covered exactly once by a unique, unexpired authenticated data record.
9. Schedule inputs share one authenticated execution payload and use exact Keccak encodings.
10. Tombstoned keys cannot re-enter; each window has an independent slashable tranche and finite release state.
11. Candidate, window, data, queue-gas, history and storage-retention work is bounded by consensus constants.
12. L1 reorg replay removes canonical state, penalties and credits from orphaned transactions together.